

mscf_ml_Homework02_solution

November 15, 2020

```
[1]: import numpy as np
import pandas as pd

#import scipy.stats
from scipy import stats
import scipy

from sklearn import linear_model
import statsmodels.api as sm
import statsmodels.formula.api as smf

from progressbar import ProgressBar

import matplotlib.pyplot as plt
import seaborn as sns
```

Problem 1

```
[2]: rho1 = 0.6; rho2 = 0.9
mu = np.array([1,2])
Sigma1 = np.matrix([[1, rho1],[rho1,1]])
Sigma2 = np.matrix([[1, rho2],[rho2,1]])
```

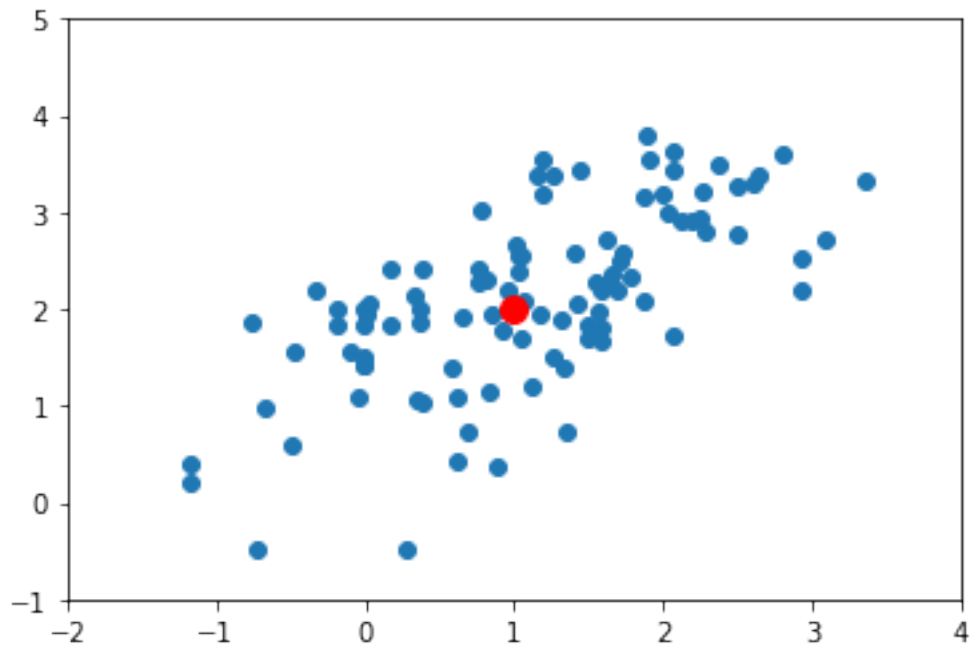
a)

```
[3]: X = np.random.multivariate_normal(mu, Sigma1, 100)
print(X.shape)
```

(100, 2)

```
[4]: plt.scatter(X[:,0],X[:,1])
plt.scatter(mu[0],mu[1], s=100, c='r')
plt.xlim([-2,4])
plt.ylim([-1,5])
```

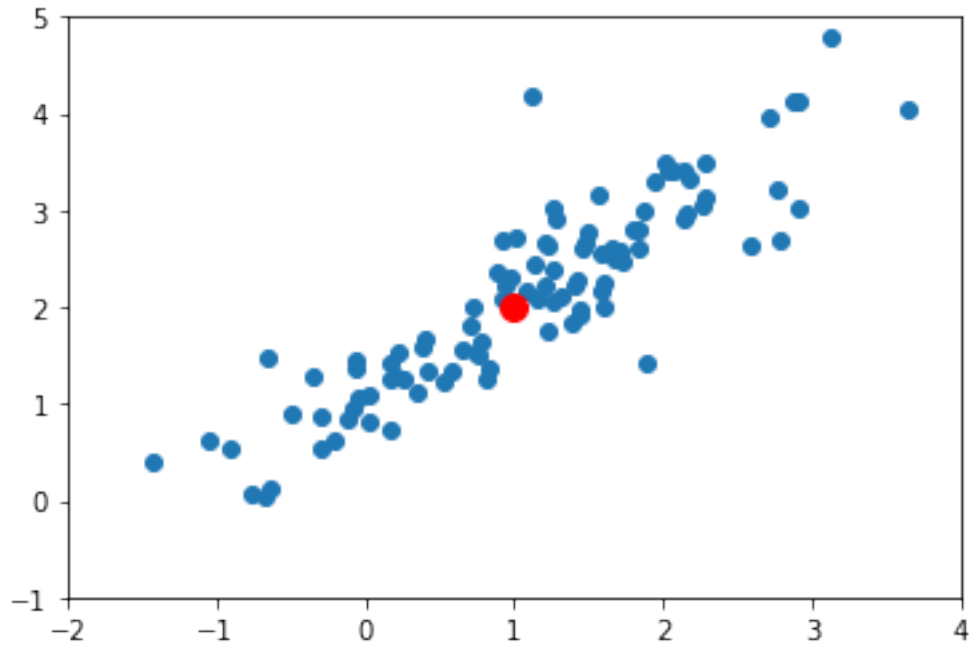
```
[4]: (-1.0, 5.0)
```



We can remake this plot for a correlation of 0.9

```
[5]: X2 = np.random.multivariate_normal(mu, Sigma2, 100)
plt.scatter(X2[:,0],X2[:,1])
plt.scatter(mu[0],mu[1], s=100, c='r')
plt.xlim([-2,4])
plt.ylim([-1,5])
```

```
[5]: (-1.0, 5.0)
```



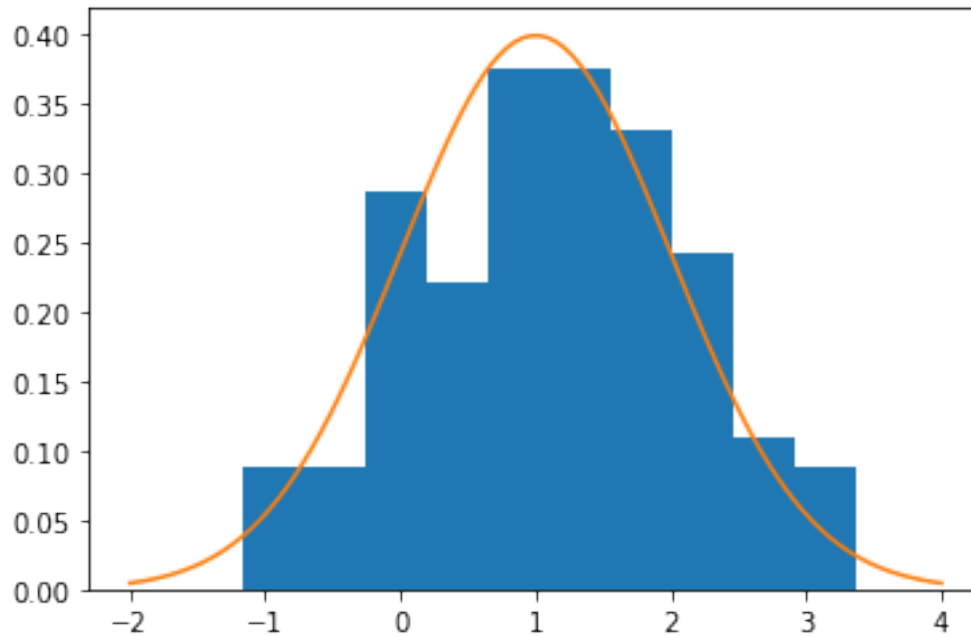
The higher correlation means the shape of the cloud is a much more elongated ellipse

b)

I'm doing everything here with a correlation of 0.6

```
[6]: plt.hist(X[:,0], density=True)
      xgrid = np.linspace(-2, 4, 100)
      dens = stats.norm.pdf(xgrid, loc=1, scale=1)
      plt.plot(xgrid, dens)
```

```
[6]: [<matplotlib.lines.Line2D at 0x7f32d8a76b80>]
```



We see that the data do look reasonably like they are $N(1,1)$

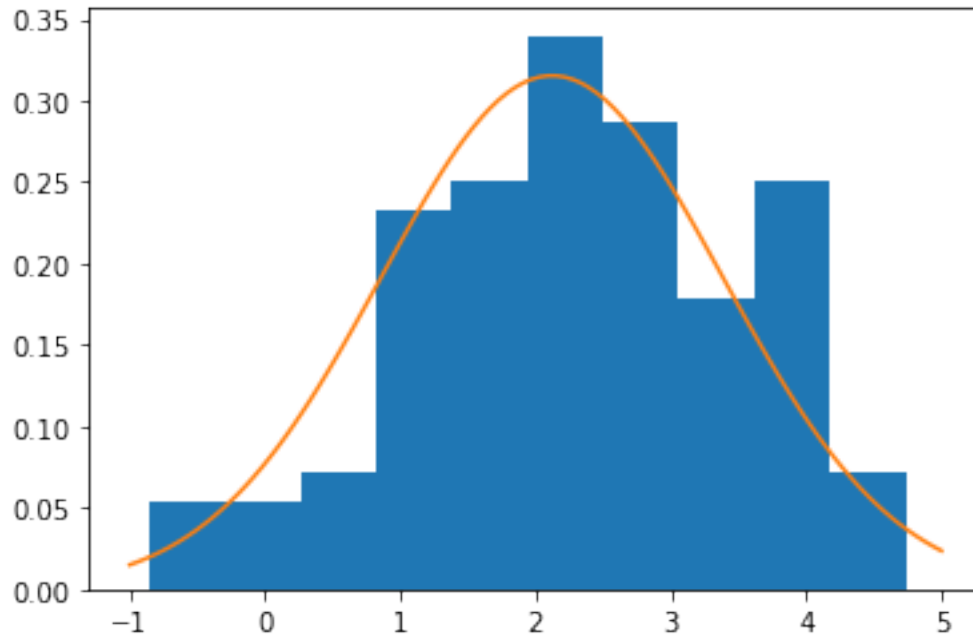
c)

```
[7]: v = np.matrix([[1/np.sqrt(2)], [1/np.sqrt(2)]])
```

```
[8]: Xv = X*v
```

```
[9]: plt.hist(Xv, density=True)
xgrid = np.linspace(-1, 5, 100)
dens = stats.norm.pdf(xgrid, loc=3/np.sqrt(2), scale=np.sqrt(1.6))
plt.plot(xgrid, dens)
```

```
[9]: [<matplotlib.lines.Line2D at 0x7f32dac4d940>]
```



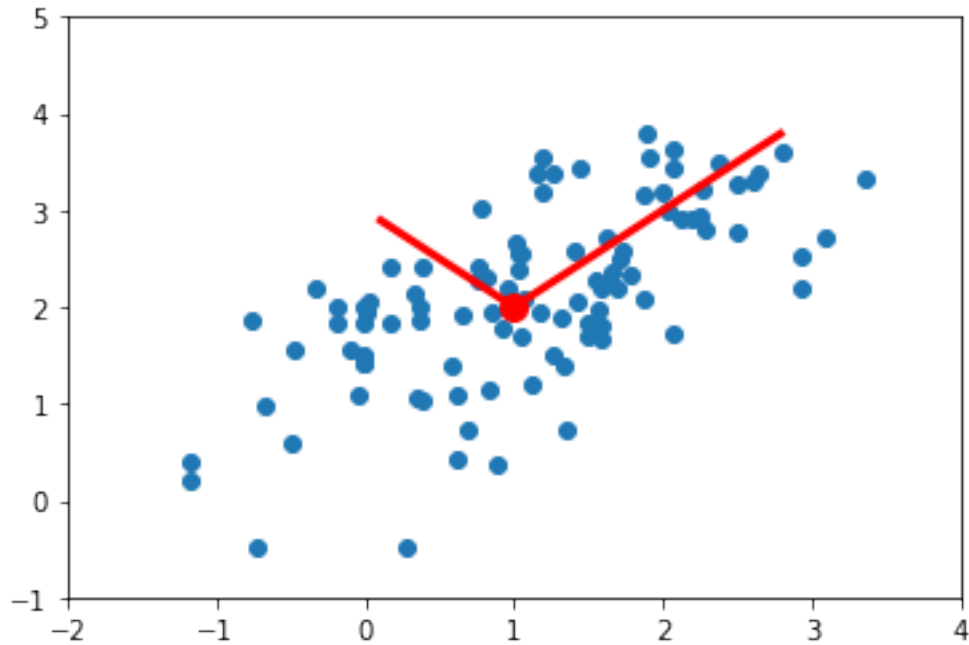
The variance should be $\sqrt{\text{Var}(Xv)} = \sqrt{v^T \Sigma v} = \sqrt{1.6} = 1.265$ (you don't have to know this) which is larger than the variance of 1 we saw before.

d)

```
[10]: evals, evecs = np.linalg.eig(Sigma1)
plt.scatter(X[:,0],X[:,1])
plt.scatter(mu[0],mu[1], s=100, c='r')
dir1 = mu + evecs[:,0].transpose()*np.sqrt(evals[0])*2
dir2 = mu + evecs[:,1].transpose()*np.sqrt(evals[1])*2
plt.plot([mu[0],dir1[0,0]], [mu[1],dir1[0,1]], c='r', linewidth=3)
plt.plot([mu[0],dir2[0,0]], [mu[1],dir2[0,1]], c='r', linewidth=3)

plt.xlim([-2,4])
plt.ylim([-1,5])
```

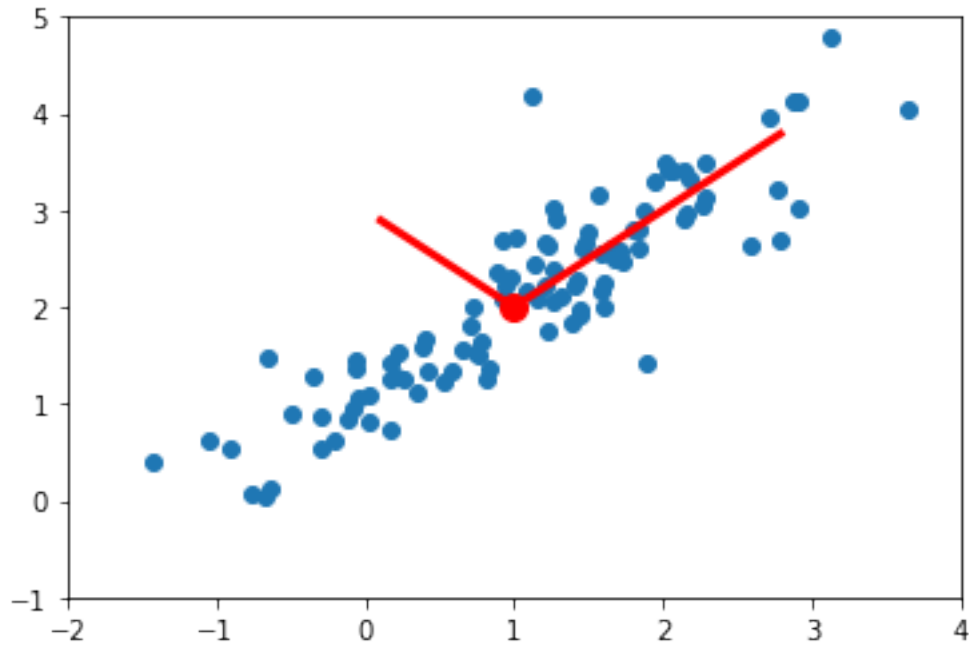
[10]: (-1.0, 5.0)



```
[11]: evals, evecs = np.linalg.eig(Sigma1)
plt.scatter(X2[:,0],X2[:,1])
plt.scatter(mu[0],mu[1], s=100, c='r')
dir1 = mu + evecs[:,0].transpose()*np.sqrt(evals[0])*2
dir2 = mu + evecs[:,1].transpose()*np.sqrt(evals[1])*2
plt.plot([mu[0],dir1[0,0]], [mu[1],dir1[0,1]], c='r', linewidth=3)
plt.plot([mu[0],dir2[0,0]], [mu[1],dir2[0,1]], c='r', linewidth=3)

plt.xlim([-2,4])
plt.ylim([-1,5])
```

```
[11]: (-1.0, 5.0)
```

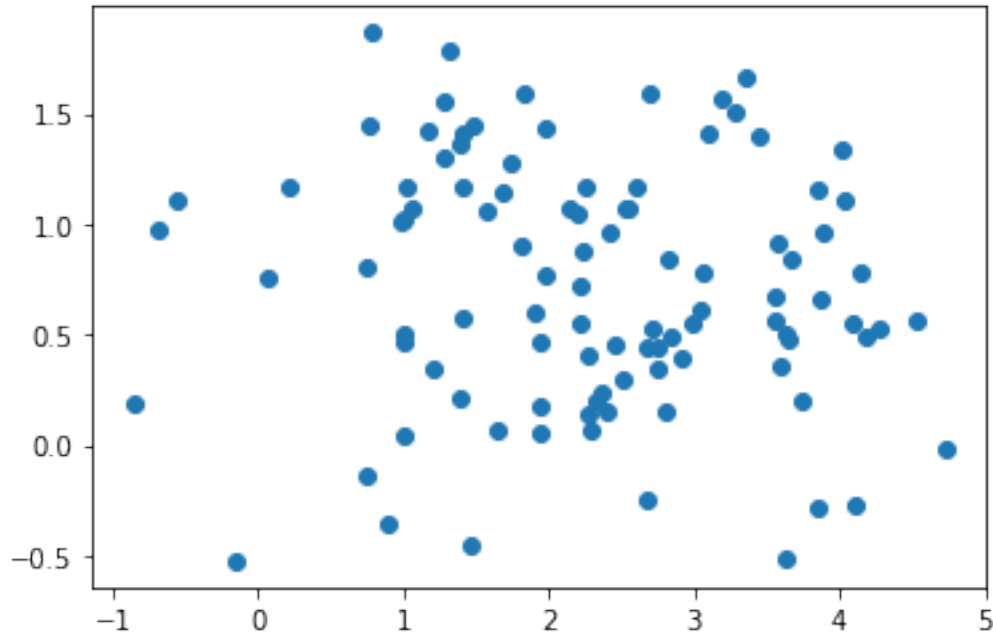


The bad aspect ratio makes it a little trickier, but the eigenvectors are orthogonal and pointing in the directions of most and least variance, respectively. Their eigenvalues are indicating the variance in those directions.

e) (though accidentally labeled g)

```
[12]: Z = np.array(np.matmul(X,evecs))  
      plt.scatter(Z[:,0],Z[:,1])
```

```
[12]: <matplotlib.collections.PathCollection at 0x7f32d8957e20>
```



```
[13]: np.corrcoef(Z, rowvar=False)
```

```
[13]: array([[ 1.          , -0.10525338],
             [-0.10525338,  1.          ]])
```

Unlike our original data, the correlation between these columns is very small (in fact, the true correlation between these random variables is zero and we are only seeing sample variation)

Problem 2

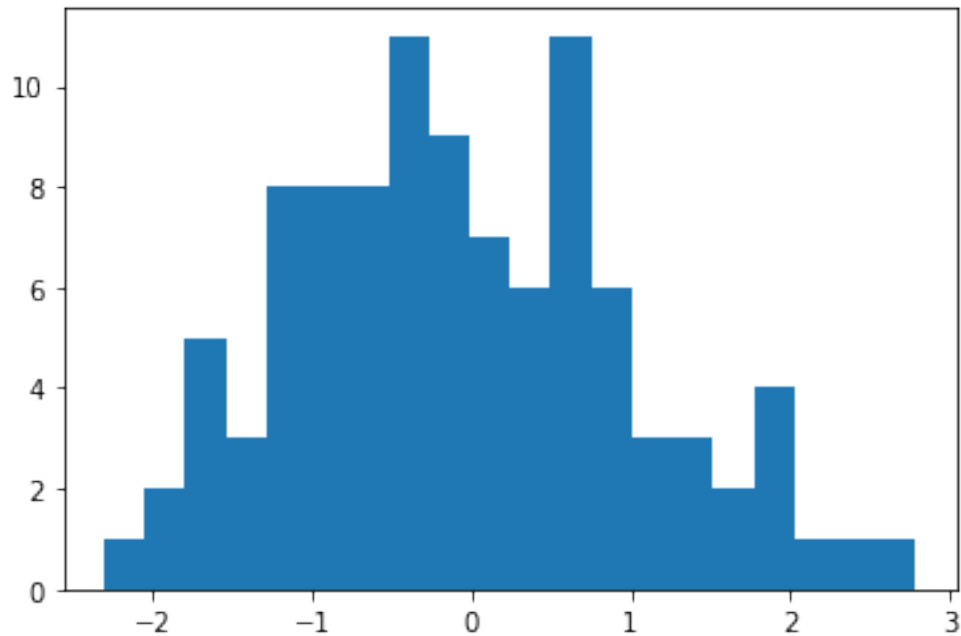
a)

```
[14]: import numpy as np
import matplotlib.pyplot as plt
```

```
[15]: reps = 100
p = 100
X = np.random.standard_normal((reps,p))
```

```
[16]: mu1hat = X[:,1]
print('Fraction of CIs that cover 0:', 1 - np.mean(mu1hat>1.96) - np.
      ↳mean(mu1hat<-1.96))
tmp = plt.hist(mu1hat,bins=20)
```

Fraction of CIs that cover 0: 0.95



We see that our intervals cover 0 95% of the time, as we hoped. The distribution of estimators is also nicely distributed around 0.

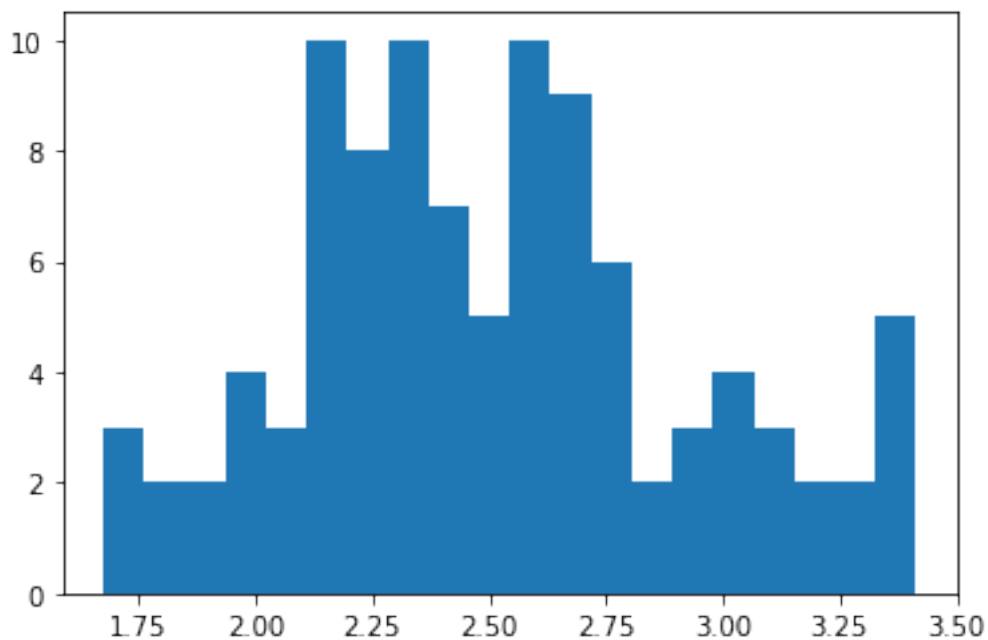
b)

```
[17]: X = np.random.standard_normal((reps,p))
```

```
[18]: #Now we just extract the biggest value from each row
      muihat = np.max(X, axis=1)
```

```
[19]: print('Fraction of CIs that cover 0:', 1 - np.mean(muihat>1.96) - np.
      ↪mean(muihat<-1.96))
      tmp = plt.hist(muihat,bins=20)
```

Fraction of CIs that cover 0: 0.06999999999999995



This is much more disturbing! Our estimates are way above 0, and less than 10% of our confidence intervals actually cover the correct number!

Problem 3

Ridge Regression

$$\hat{\beta}^{\text{ridge}} = \operatorname{argmin}_{\beta \in \mathbb{R}^p} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

a)

Get the ridge equation into a format for a linear regression optimization with $\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}$, and $\tilde{X} =$

$$\begin{bmatrix} X \\ \sqrt{\lambda}I \end{bmatrix}.$$

** Solution **

$$\begin{aligned}
\hat{\beta}^{\text{ridge}} &= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} (Y - X\beta)^T (Y - X\beta) + \lambda \beta^T \beta \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} (Y - X\beta)^T (Y - X\beta) + \sqrt{\lambda} \beta^T \beta \sqrt{\lambda} \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} (Y - X\beta)^T (Y - X\beta) + \sqrt{\lambda} \beta^T I^T I \beta \sqrt{\lambda} \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} (Y - X\beta)^T (Y - X\beta) + (\sqrt{\lambda} I \beta)^T (\sqrt{\lambda} I \beta) \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} (Y - X\beta)^T (Y - X\beta) + (-\sqrt{\lambda} I \beta)^T (-\sqrt{\lambda} I \beta) \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} (Y - X\beta)^T (Y - X\beta) + (0 - \sqrt{\lambda} I \beta)^T (0 - \sqrt{\lambda} I \beta) \\
&= \operatorname{argmin}_{\beta \in \mathbb{R}^p} \left(\begin{bmatrix} Y \\ 0 \end{bmatrix} - \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix} \beta \right)^T \left(\begin{bmatrix} Y \\ 0 \end{bmatrix} - \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix} \beta \right)
\end{aligned}$$

As such, $\hat{\beta}^{\text{ridge}}$ can be seen as the solution of a linear regression with $\tilde{y}$ and $\tilde{X}$.

b)

Show that with $\lambda > 0$ the matrix $\tilde{X}$ is full rank.

**** Solution ****

Notice that each column has only 1 of the last p rows with non-zero values. As such no linear combination would let any set of columns add up to a column not in the set. This means all columns a $\tilde{X}$ are independent, making $\tilde{X}$ full rank.

c)

Write explicit formulat for $\hat{\beta}^{\text{ridge}}$ involving X, y, λ .

**** Solution ****

Using **** part a****, and recalling that $\hat{\beta}^{\text{OLS}} = (X^T X)^{-1} X^T Y$:

$$\begin{aligned}
\hat{\beta}^{\text{ridge}} &= (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T \tilde{Y} \\
&= \left(\begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix}^T \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix} \right)^{-1} \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix}^T \begin{bmatrix} Y \\ 0 \end{bmatrix}
\end{aligned}$$

where we observe that 1. $\begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix}^T \begin{bmatrix} Y \\ 0 \end{bmatrix} = X^T Y$ 2. $\begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix}^T \begin{bmatrix} X \\ \sqrt{\lambda} I \end{bmatrix} = X^T X + \lambda I$

Which means that

$$\hat{\beta}^{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T Y$$

Problem 4

I don't have this one written up yet to share, but conveniently it's essentially on page 66 of ESL, reproduced below:

a)

Consider least squares linear regression. We know that $\hat{Y} = X\hat{\beta} = X(X^T X)^{-1} X^T Y$. Replace X with its SVD, and simplify as much as possible. How can you interpret this in terms of the columns of U and/or V ?

Substituting in $X = UDV^T$, we have

$$\begin{aligned}\hat{Y} &= UDV^T(VDU^TUDV^T)^{-1}VDU^TY \\ &= UDV^T(VD^2V^T)^{-1}VDU^TY \\ &= UDV^TVD^{-2}V^TVDU^TY \\ &= UU^TY.\end{aligned}$$

Since U is an orthogonal matrix, UU^T is the projection operator onto the column span of U . This means that the fitted value $\hat{Y}$ is just the projection of Y onto the column span of U . (Note: It will turn out that the columns of U are just a basis for the column span of X , so this fits with our previous interpretation of linear regression.)

b)

Now consider ridge regression with fixed λ . Again, replace X with its SVD and simplify as much as possible. You will wind up with a matrix product that looks more complicated than in (a).

Substituting, we obtain

$$\begin{aligned}\hat{Y} &= X(X^T X + \lambda I)^{-1} X^T Y \\ &= UDV^T(VDU^TUDV^T + \lambda I)^{-1}VDU^TY \\ &= UDV^T(VD^2V^T + \lambda I)^{-1}VDU^TY \\ &= UDV^T(VD^2V^T + \lambda VIV^T)^{-1}VDU^TY \\ &= UD(D^2 + \lambda I)^{-1}DU^TY\end{aligned}$$

We can't simplify this further in matrix form.

c)

Try rewriting the product from (b) as a sum. Compare this to (b).

We can write the solution to part (b) as a sum, as

$$\sum_{j=1}^p \frac{d_j^2}{d_j^2 + \lambda} u_j u_j^T Y$$

Note that we could write the solution to part (b) similarly as $UU^TY = \sum_{j=1}^p u_j u_j^T Y$. When we compare, we see that ridge regression is shrinking each term in this sum by a factor of $\frac{d_j^2}{d_j^2 + \lambda}$. When

$\lambda = 0$, this factor is 1. As λ increases, this shrinkage factor gets smaller, causing greater shrinkage. We also see that the shrinkage effect is greater on terms that have smaller d_j .

(Note: You'll eventually see PCA in data science and in ML, if you haven't seen it elsewhere. It will turn out that these d_j^2 are proportional to the variance the PCA components. The interpretation will then be that ridge regression shrinks the dimensions with small eigenvalues in the covariance matrix of X , which is rather pretty.)